The class `GridCollection`

Manual (draft)

Alex Pfaff
nonintersective@gmail.com

December 27, 2025

Contents

Welcome to Sudoku and Congratulations on your acquisition of `GridCollection`! – it reveals your good taste in sophisticated cool stuff, highlights your fearlessness in the face of structural insanity and combinatorial explosions, and thus betrays your non-standard curiosity towards the unknown (depths of Sudoku geometry); well done!

`GridCollection` is a class that manages collections of Sudoku grids, it allows you to "activate" various categories of sets of Sudoku grids (e.g. permutation series), encode or manipulate them in various ways, and create (train/test) datasets for various tasks, notably, to train ML models / neural networks (even though the original purpose of this endeavour is more academic in nature: Sudoku as a formal language – the grammar of Sudoku grids). I will occasionally comment on some such practical practical aspects. One noteworthy feature is the option to manufacture your own (customized) garbage collection; e.g. for classification tasks, you can determine certain properties of the data that will receive the label `False` – awesome, right?

This manual illustrates the most important features / uses of the `GridCollection` class; there is also some extensive documentation in the code (docstring, use the Python built-in function `help()` to access the documentation). Feel free to drop me a line (feedback, questions, bug reports, suggestions, comments on Sudoku geometry ...).

Best wishes, Alex

PS: this manual is "work in progress" to use an academic phrase; certain relevant aspects have not been covered yet. Be patient, or – as already stated above – feel free drop me a line.

⇒ **Make sure** you have downloaded from the repository at least

- `PsQ_Grid.py`
- `PsQ_GridCollection.py`
- `utilFunX.py`

into the same folder. In order to get started, run `PsQ_GridCollection.py` or import it into you favourite IDE via

```
>>> from PsQ_GridCollection import GridCollection
>>> # suggested convention: ... import GridCollection as GC
```

1 Initialization and Central Properties

1.1 Constructor

The class `Grid` has a method – `permuteGrids(toCollection = True)` – that generates all 1679616 grid permutations from the current grid ($\rightarrow$ the vertical series), and, by default, returns a `GridCollection` object. Likewise, the method `get_minTripletCollection(to_gc=True)` returns a `GridCollection` object.

The class `GridCollection` itself also provides a range of methods for initialization:

1.1.1 Proper Constructor

```
>>> gc = GridCollection(gridArray)
>>> # with gridArray.shape == (n, 81) or (n, 9, 9); n = number of grids
>>> # with gridArray.dtype == int or 'U1' (str)
```

This method presupposes that the user has already a suitable collection of (valid!) Sudoku grids (as np.ndarray) of an appropriate shape and dtype. Moreover, the user bears responsibility for the validity of the grids contained! (see Sect. 1.3)

1.1.2 Static constructor method (basis: vertical series)

```
>>> gc = GridCollection.from_scratch()
Generating 1296 X 1296 Grid Permutations: 100%| 1296/1296

>>> print(gc)
GridCollection[
                internal shape: (1679616, 81)
                active series: not_activated
                   size: 0
                labeled series: 0
                labels: False
                  size: 0
                garbage: False
                  total size: 0
                    guest_grids              : 0
                    false_fromCurrent_seq    : 0
                    false_fromCurrent_switch : 0
                    false_cardinality        : 0
                    false_off_by_X           : 0
                    false_arbitrary          : 0
                one_hot encoding: False
                internal abc collection: False
              ]
```

This method generates a (valid) Sudoku grid g at random, and then internally calls `g.permuteGrids()`. Thus **the default grid collection is a collection of grid permutations** (= vertical permutation series). The method allows the option to initialize a specific grid via the setting `GridCollection.from_scratch(grd = myGrid)` where `myGrid` is a valid Sudoku grid (as array or sequence type: tuple, list, string).

1.1.3 Static constructor method (basis: sql database)

```
>>> gc = GridCollection.from_sql(db_name, how_many = None)
```

This method presupposes a suitable/valid grid collection stored in an sql database by an appropriate method (see Sect. XXX). `db_name` specifies the name of the db/file, and `how_many` specifies how many grids are to be loaded; plausibly, the number specified must not be greater than the size of the database. If set to `None` (default), the entire database is loaded into the `GridCollection` object.

1.1.4 Static constructor method (basis: chunks of vertical series)

```
>>> gc = GridCollection.from_scratch_div(how_many = 500000,
                                         series = 10,
                                         seed = 42)
Generating guest grids, series 1:
Generating 1296 X 1296 Grid Permutations: 100%|| 1296/1296
Diaflecting grid collection: 100%|| 1679616/1679616
Current size: 50000
Generating guest grids, series 2:
Generating 1296 X 1296 Grid Permutations: 100%|| 1296/1296
Diaflecting grid collection: 100%|| 1679616/1679616
Current size: 100000

. . . . . .

Generating guest grids, series 9:
Generating 1296 X 1296 Grid Permutations: 100%|| 1296/1296
Diaflecting grid collection: 100%|| 1679616/1679616
Current size: 450000
Generating guest grids, series 10:
Generating 1296 X 1296 Grid Permutations: 100%|| 1296/1296
Diaflecting grid collection: 100%|| 1679616/1679616
Current size: 500000
Checking for duplicates: 100%|| 500000/500000
Generating missing grids: 100%|| 3359232/3359232
Adding missing grids: 100%|| 3699/3699
```

```
>>> print(gc)
GridCollection[
                 internal shape: (500000, 81)
                 active series: not_activated
                 size: 0

                 . . . .

               ]
```

A diversified version of static method `.from_scratch` ensuring the collection contains (random) samples from different vertical series. Parameters:

- `how_many`: specifies the size of the prospective grid collection (i.e. number of grids).
- `series`: specifies the number of vertical series that are generated; from every series, $\lfloor \frac{how\_many}{series} \rfloor$ randomly selected grids are contributed to the prospective collection.

At the end, the method checks if duplicates have been generated (accidentally), removes them; finally, the methods checks the actual size of the current collection, compares it to the parameter `how_many`, an generates new grids until size `how_many` is reached.
NB$_1$: make sure that $\lfloor \frac{how\_many}{series} \rfloor < 1,679,616$ (= the size of a vertical series); in fact, make sure it's quite a bit smaller in order for the method to work properly (with too little leeway, it is possible that the method returns less than `how_many` grids – rarely, but it can happen).
NB$_1$: increasing the values for `series` increases the diversity (greater number of vertical series as basis), but it also increases runtime.

1.1.5 Static constructor method (basis: recodings of constrained grids)

```
GridCollection.from_scratch_constraint(how_many: int = 250000,
                                       anti_knight=False,
                                       modular_knight=False,
                                       diagonal_choice=None,
                                       seed_batch=5,
                                       max_expansion_per_seed=15000)
```

1.2 Attributes

As illustrated above, the current status of the collection object can be displayed via print statement `print(gc)`, alternatively, via the method `gc.getInfo()`. Various pieces of information can be accessed individually via the respective attributes:

```
>>> gc.BASE_NUMBER
3
>>> # 3 is the basenumber for classical Sudoku grids;
>>> # presupposed for most functionalities
>>> gc.DIMENSION  # = gc.BASE_NUMBER^2
9
>>> gc.SIZE_grid  # = gc.DIMENSION^2
81

>>> gc.SIZE_collection          # grid count: internal collection
>>> gc.SIZE_activeSeries        # grid count: active series
>>> gc.SIZE_garbageCollection   # grid count: garbage collection
>>> gc.SIZE_labels              # size of the 'labels' dataset
>>> gc.classes                  # number of distinct classes
                                # in the 'labels' dataset
                                # -> actual classes @multiclassSeries;
                                # -> k @puzzleSeries (see below)

>>> gc.gc.activationStatus      # name of active series, if activated,
                                # 'not_activated' otherwise

>>> # the following return (a copy) of actual datasets
>>> gc.collection        # internal collection
                         # from which the object was created
>>> gc.activeSeries      # currently activated collection
>>> gc.labels            # collection of labels for gc.activeSeries
                         # empty unless suitable series is activated
>>> gc.garbageCollection # current collection of garbage grids

>>> gc.getGarbageStatus


  ......



>>> clear_all() # resets all activations
```

1.3 Checking methods

(coming soon)

2 Activation → "Active Grids"

The internal collection (from which the object was initialized) is inert, as it were; it needs to be *activated*.[1] Based on the `activeSeries`, various manipulations can be conducted and datasets (including garbage collections) can be built. Therefore, activation is the crucial next step after initialization; the class `GridCollection` provides a range of different activation methods to be illustrated below (notice, that in some cases, the garbage collection must be initialized before activation!).

2.1 Vertical Series

```
>>> gc.activate_verticalSeries()

>>> print(gc)
GridCollection[
                internal shape: (1679616, 81)
                active series: vertical
                size: 1679616

                . . . . .

               ]
```

If the object is initialized via `gc = GridCollection.from_scratch()` (or e.g. `gc = g.permuteGrids()`), the internal collection **is** a vertical series, and will be activated directly (see fn. 1).

If the object is initialized differently and the internal collection is not a collection of (1679616) grid permutations, an exception is raised suggesting two possible fixes:

```
>>> # produces collection of triplet grids, not row/col permutations!
>>> gc = g.get_minTripletCollection(to_gc=True)
>>> print(gc)
GridCollection[
                internal shape: (373248, 81)
                ....

>>> gc.activate_verticalSeries(explicit_validation=True) # default setting
...
ValueError: Collection does not resemble a vertical series.
Use `.activate_thisSeries()` instead
-- or specify 'new_series_fromIndex' parameter.
```

[1] The original purpose of the class `GridCollection` was to manage large collections of grid permutations. For this reason, vertical and horizontal permutation series still play a major role in various activation and generation procedures / methods (this will be externalized and more delegated in later versions).

If the user wants to activate the internal collection nonetheless (i.e. regardless of provenance / contents), there is a designated method doing just that; see Sect. XXX

If the user insists on activating a vertical series, however, the method provides a parameter $0 \leq$ `new_series_fromIndex` $<$ `gc.SIZE_collection`, that takes the specified grid from the internal collection, and generates a vertical series from scratch $\rightarrow$ from which, in turn, the active series initialized:

```
>>> gc.activate_verticalSeries(new_series_fromIndex=42)
Generating 1296 X 1296 Grid Permutations: 100%|| 1296/1296

>>> print(gc)
GridCollection[
                  internal shape: (373248, 81)
                  active series: vertical_new
                  size: 1679616
                 . . . .
                ]
```

2.2 Horizontal Series

Differently from the vertical permutation series, which is the (exhaustive) collection of grid permutations obtained from row/column permutations, the horizontal permutation series is obtained from label permutations; this is achieved by exhaustive label recoding. Any classical 9×9 Grid can be recoded in 9! = 362,880 ways, and all those grids translate to the same abc-grid, i.e. they have the same (deep) distribution. The activation method takes an index argument (default: `idx = 0`), with $idx \leq 0 <$ `gc.SIZE_collection`, which in turn, picks the corresponding grid from the internal collection that serves as the basis for the horizontal permutations:

```
>>> gc.activate_horizontalSeries(idx = 666) # grid 666 -> basis

>>> print(gc)
GridCollection[
                  internal shape: (1679616, 81)
                  active series: horizontal
                  size: 362880
                  . . .
                ]
```

2.3 THIS Series

If the internal collection does not constitute a vertical series, but the user wants to activate it, THIS is the method of your choice:

```
>>> gc.activate_thisSeries(name = 'THIS')
```

```
>>> print(gc)
GridCollection[
                internal shape: (1679616, 81)
                active series: THIS
                size: 1679616

                . . . .

              ]
```

'THIS' is the default label, but obviously, it can customized (`name = 'my_series13'`); (`name = 'my_lucky_grids'`), ideally in a way that actually says something about the provenance/properties of the collection. A suitable use for the scenario mentioned in Sect. 2.1 is illustrated below:[2]

```
>>> gc = g.get_minTripletCollection(to_gc=True)
>>> gc.activate_thisSeries(name='3-triplets')

>>> print(gc)
GridCollection[
                internal shape: (373248, 81)
                active series: 3-triplets
                size: 373248
               . . . .
             ]
```

Likewise, when using the diverse constructor, the following labeling may be useful:

```
>>> gc = GridCollection.from_scratch_div()
. . . .
>>> gc.activate_thisSeries(name='diverse')  # or name = 'random' etc.

print(gc)
GridCollection[
                internal shape: (500000, 81)
                active series: diverse
                size: 500000

               . . . .

              ]
```

NB: the class `GridCollection` also has a method `activate_randomSeries()` that, in essence, does the same thing. But it should be considered deprecated and will be removed in a future version.

[2]For more information on *triplets*, see "Grid_class_manual", Sect. 3.2.

2.4 Multi-Class Series

For multiclass classification (and other creative) tasks, this method is your friend. A few words are in order, however, about some noteworthy peculiartities. Firstly, classes are constituted by horizontal series (based on a random selection of grids from the internal collection), meaning that each class comprises exactly 362,880 members. In the future it will be possible base classes on other features and (geometric) properties, but notice that any class(ification) of grids that can reasonably justified will be a finite set and the procedure will generate an exhaustive dataset. This is the motivation, secondly, for the inclusion of a garbage class = everything that is not a member of our actual classes. The method has the recommended default setting `(make_garbage=True)`, which therefore and thirdly presupposes that the garbage collection has been initialized to an appropriate size (min: 362,880; the garbage collection will be discussed in detail in Sect. CCC):

```
>>> # if garbage size < 362880, an exception will be raised
>>> gc.makeFalseGrids_cardinality(how_many=362880)

>>> gc.activate_multiClassSeries(classes=5)

>>> print(gc)
GridCollection[
                internal shape: (500000, 81)
                active series: multiclass
                size: 2177280
                labeled series: 6 classes
                labels: True
                  size: 2177280
                garbage: True
                   total size: 362880
                     guest_grids              : 0
                     false_fromCurrent_seq    : 0
                     false_fromCurrent_switch : 0
                     false_cardinality        : 362880
                     false_off_by_X           : 0
                     false_arbitrary          : 0
                one_hot encoding: False
                internal abc collection: False
              ]
```

The string representation finally reveals the fourth peculiarity: even though 5 classes are specified in the activation method (`classes=5`), we find `labeled series: 6 classes`. This discrepancy stems from the inclusion of the garbage class; therefore (size) 2177280 = 6 (classes) $\times$ 362880. The garbage class always receives the label 0, whereas the actual classes receive the labels 1–5 (or whatever number specified) in an arbitrary fashion (it's hard to see what meaningful labels should be assigned

otherwise).

2.5 Puzzle Series

The puzzle series constitutes a dataset suitable to train a (DL) model to solve Sudoku grids (and other creative tasks). More specifically, it creates a dataset on the basis of the currently active series. Here is the method signature:

```
activate_puzzleSeries(k: int,
                      extra_high_vals: bool = True,
                      include_parentSet: bool = True,
                      run: int = 3
                      ) -> None
```

The only mandatory parameter is k, which specifies the (maximum) number of blanks; put differently, $81 - k =$ number of given digits, indicating the degree of difficulty of the grids to be solved. Let's elaborate on that point: since the required minimum number of give digits is 17, it is not recommended to set $k > 64$. Moreover, regardless of what machines and models can actually do, one does not play $\rightarrow$ solve a Sudoku grid by filling in all the blanks in one go, where would be the thrill in that? No, we proceed cautiously and cunningly one cell at a time. For this reason, k is not a rigid parameter, but an upper bound; the underlying algorithm generates an equal number of grids with $n = 1$ blank, $n = 2$ blanks ... $n = k - 1$ blanks, $n = k$ blanks, respectively. For a given n and a given grid currently being iterated over, the positions for the n blanks are randomly selected.

In order to see what a data point (blanked grid, label) looks like, let's first do some actual activation:

```
>>> gc.activate_puzzleSeries(k=55)
Generating grids with 1-55 blanks: 100%||
Generating grids with 35-55 blanks: 100%||

>>> print(gc)
GridCollection[
                  internal shape: (373248, 81)
                  active series: k_55_max_puzzle_from_3-triples
                     size: 2612736
                  labeled series: 55 k_blanks
                  labels: True
                     size: 2612736

                  . . .

                ]
```

```
>>> # active series is now a collection of blanked grids
>>> # with n = 1, 2..., k blanks
>>> # blanks are represented by zeros
>>> gc.activeSeries[654321].reshape(9,9)

array([[0, 2, 3, 0, 5, 6, 0, 0, 9],
       [0, 6, 4, 8, 9, 7, 0, 3, 1],
       [7, 0, 0, 2, 3, 1, 5, 6, 4],
       [3, 1, 2, 0, 0, 4, 9, 7, 0],
       [6, 0, 5, 7, 8, 9, 0, 0, 2],
       [8, 9, 0, 3, 1, 2, 0, 4, 0],
       [2, 3, 1, 0, 0, 0, 8, 9, 7],
       [4, 5, 6, 9, 7, 8, 0, 2, 0],
       [9, 7, 8, 1, 0, 3, 0, 5, 6]], dtype=uint8)


>>> # the labels are complete grids with no blanks!
>>> # the following is the label for the blanked grid above
>>> gc.labels[654321].reshape(9,9)

array([[1, 2, 3, 4, 5, 6, 7, 8, 9],
       [5, 6, 4, 8, 9, 7, 2, 3, 1],
       [7, 8, 9, 2, 3, 1, 5, 6, 4],
       [3, 1, 2, 5, 6, 4, 9, 7, 8],
       [6, 4, 5, 7, 8, 9, 3, 1, 2],
       [8, 9, 7, 3, 1, 2, 6, 4, 5],
       [2, 3, 1, 6, 4, 5, 8, 9, 7],
       [4, 5, 6, 9, 7, 8, 1, 2, 3],
       [9, 7, 8, 1, 2, 3, 4, 5, 6]], dtype=uint8)
```

As shown, a label is not simply a list (vector) of the missing values for the blanks, but a complete grid. The problem with the *label = list-of-missing-values strategy* is that it entails a specified output size (= `len(label)`). This, in turn, is at odds with the variable parameter $n = 1, 2...k$: we would need k different datasets of different sizes and train k different models, one for each value n. So in spite of a slight overhead, the alternative of using a complete grid (the "solution" to a blanked grid") as a label is a pretty nifty and elegant way to avoid a logistic and computational nightnmare :)

Back to the parameters: `extra_high_vals` is by default set to `True`; in this case an additional set of blanked grids is generated for $n = \lfloor \frac{2k}{3} \rfloor ... k-1, k$. Intuitively, grids with a larger number of blanks tend to be more "difficult/complex" than grids with a smaller number of blanks, at least combinatorially: for $n = 1$, there are 81 possible placements for the one blank, but with increasing n, the number of possible (combinations of) placements grows super-exponentially: $\binom{81}{n}$. In other words, for larger n, there will be less and less^n actual occurrences of possible placements of blanks in the dataset. This parameter is a modest attempt to slightly ameliorate this

combinatorial inequality.

The parameter `include_parentSet`: if set to `True` (default), the entire (original) active series is added to the puzzle series, with 0 blanks; in these cases, the "blanked" grids are identical to their respective labels.

The parameter `run` specifies the number of iterations over the currently active series; the size of the puzzle series is thus

$$
\begin{aligned}
&run \times \texttt{gc.SIZE\_activeSeries} \\
&\quad \times 2 \text{ for } \texttt{extra\_high\_vals} \\
&\quad + \texttt{gc.SIZE\_activeSeries}, \text{ if } \texttt{include\_parentSet=True}
\end{aligned}
$$

The following simplified code snippet illustrates what happens during a run:

```
>>> for _ in range(run):
        for idx in range(gc.SIZE_activeSeries):
            n = idx % k + 1  # -> values from 1 to k
            grid = gc.activeSeries[idx]
            labels.add(grid)
            # modify the grid:
            # select n positions in grid at random
            # -> replace the actual n values with 0:
            # grid[row_n, col_n] = 0
            ....
            puzzles.add(grid)
            ...
```

Every grid from the original active series occurs run times as a blanked grid in the puzzle series. It is conceivable that the same grid occurs run times with the same number of n blanks (unlikely in the same positions!), but more likely, with different values n. But even if $run \geq k$, it is very extremely unlikely that the same grid occurs with all values $n \in [k]$, and it's virtually impossible that the puzzle series contains a complete, consecutive solution path for even a single grid (from k to 0 blanks).

Obviously, increasing run leads to more diversity in the dataset (possible combinations of blank positions in the grid), but even more obviously, it leads to a a larger dataset. This has various implications for the runtime on a household computer. BUT consider this: this method allows you to **create an insanely large high-quality labeled dataset**. Obviously the kind and size of the current active series do play a role, but the range of combinatorial possibilities almost guarantees that the dataset won't get repetitive no matter how much you increase run.

Remember: Only one single grid gives rise to $\sum_{n=1}^{k} \binom{81}{n}$ blanked grids!

Brief schematic of a solving strategy: best argmax wins! Once the model is trained, a prediction on a given grid with k blanks returns one complete (not necessarily valid!) grid with 81 values. Since we want a plausible Sudoku solve, i.e. one cell at a time, we will regard 80 of the predicted values; firstly, discard all predictions for positions where

the input grid has a digit, only consider positions where the input grid has a blank (i.e. 0). Out of these, select the one with the highest probability, e.g. the predicted probability for position a to contain a 7 is 0.99, for position b to contain a 3 is 0.95, for position c to contain a 5 is 0.87 ... we pick the prediction for position a and insert 7 in that position in the blanked grid. In other words, only 1 out 81 predictions is chosen; I have termed this strategy *Best Argmax wins!*. The following procedure is recursive: the model makes a prediction for the grid with $k-1$ blanks, pick the best argmax, insert the value, and repeat – until the (originally) blanked contains only digits (i.e. no zeros) $\rightarrow$ success!, or until the model makes a false prediction (meaning that the best argmax was not good enough, the predicted value violates the Sudoku constraints) $\rightarrow$ failure!

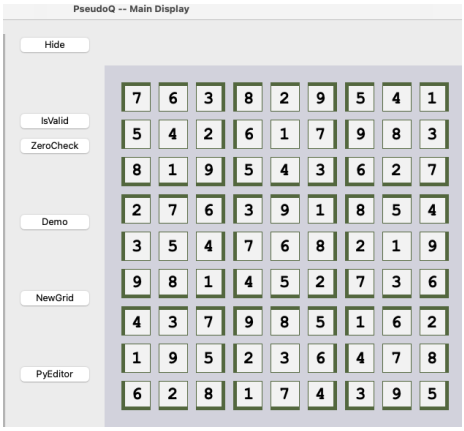
The ideal scenario is depicted schematically below (not to be taken literally since we ignore some internal steps like reshaping, selecting the best argmax, inserting the value etc.); we start out from a grid `grid_k_blanks` as first input

```
>>> grid_k_minus_1_blanks = model.predict(grid_k_blanks)
>>> grid_k_minus_2_blanks = model.predict(grid_k_minus_1_blanks)
>>> grid_k_minus_2_blanks = model.predict(grid_k_minus_2_blanks)
>>> grid_k_minus_3_blanks = model.predict(grid_k_minus_3_blanks)
>>> . . . . .
>>> grid_2_blanks         = model.predict(grid_3_blanks)
>>> grid_1_blank          = model.predict(grid_2_blanks)
>>> solved_grid           = model.predict(grid_1_blank)
```

An implementation of one useful CNN model can be found in *PsQ_Solver_CNN.py* and *psq_solver_player.ipynb* in the repository. The notebook[3] moreover illustrates the best argmax strategy in more detail with a concrete blanked grid.

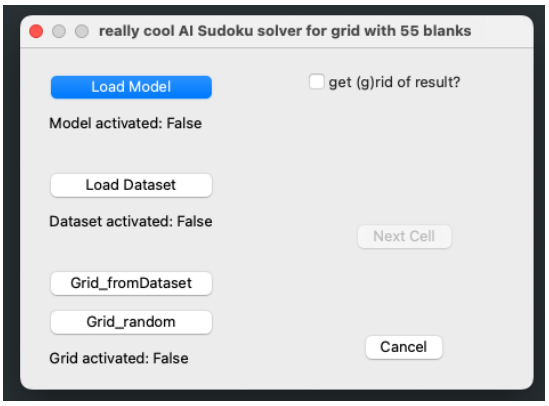
Finally, a more dynamic illustration in the GUI is found in *PsQ_Solver_CNN.py*

- run the file; click "Demo" in the main window

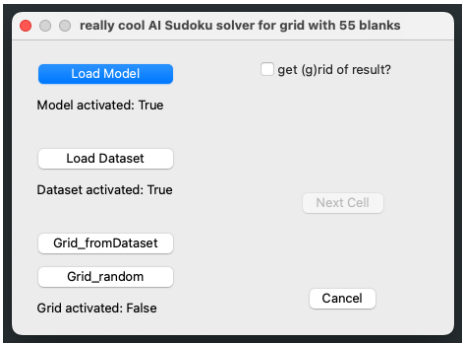


- enter a value for k, and the following dialogue opens:

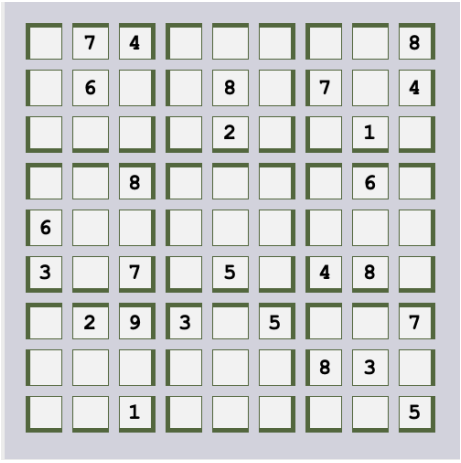
[3] `https://github.com/A-Lex-McLee/PseudoQ-2.1/blob/main/psq_solver_player.ipynb`

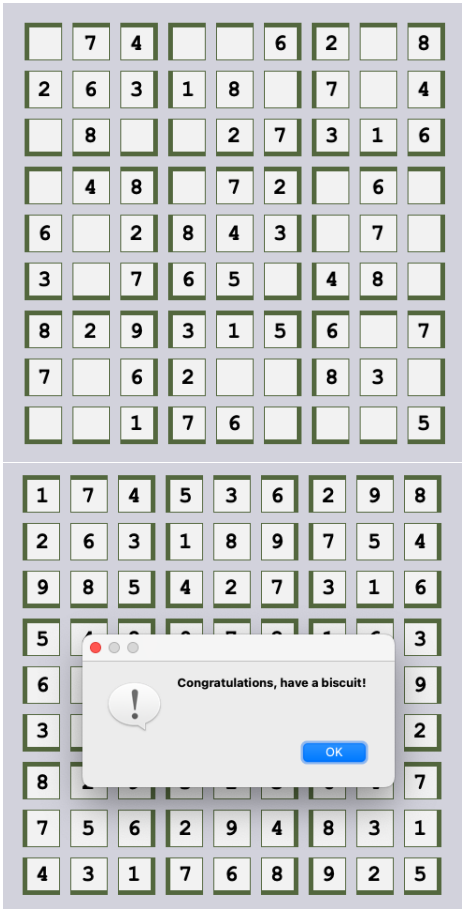


- load model: *solver1.keras*; load dataset: *reduced_dataset_1.npy* (both available in the repository):



- click "Grid_fromDataset" ... and: play! click "Next Cell" until success or failure:





$\Rightarrow$ success rate: $> 90\%$ – not bad!
but if you try a grid not from the dataset ("Grid_random"), the failure rate increases drastically $\rightarrow$ you need a better model (more runs, greater dataset ... different model architecture?)

PS: it's not a violation of any rules if you want to create a puzzle series only to solve them yourselves – good luck and enjoy ;)
. . . one CAVEAT, though: it is not guaranteed that every blanked grid really is a puzzle with a unique solution! one simple example: if $k \geq 18$ and the missing values include $\{4\}^9$ and $\{7\}^9$ (all 4s and 7s are missing), there are **two** solutions to the grid!

3 The Garbage Collection

Finally! Let's talk $Garbage$ (NB: not trash).
The class `GridCollection` provides a range of mechanisms to create carefully curated, highly diversified *Garbage*, stored in `gc.garbageCollection`. Such a garbage collection turns out to be particularly useful for classification tasks, the most basic of which is validity: is a given grid valid (in the Sudoku sense) or not?

For a simple binary classification setup, the current active series receives the label 1 (valid), while the garbage collection receives the label 0 (invalid). Together, these form a dataset on which a model can be trained.

Before proceeding, it is worth pausing for scale. There are approximately 6.7×10^{21} valid Sudoku grids, whereas there are $9^{81} \approx 1.9 \times 10^{77}$ possible assignments of digits 1–9 to an 81-cell grid. In other words: the universe of false grids is not just larger, it is *astronomically* larger. Within this vast space, we can distinguish two broad classes of invalid grids:

- **Cardinality violation**: A valid grid contains each of the digits 1–9 exactly nine times; any deviation from this immediately yields an invalid grid. Such deviations come in various degrees:
  (i) minor deviations: for example, if some digit a occurs ten times, then necessarily some digit b occurs only eight times (the total count of 81 digits must still be satisfied). Variants include a occurring eleven times while b and c occur eight times, and so on.
  (ii) intermediate cases: one digit does not occur at all; for instance, seven digits occur eleven times and one digit occurs four times (only eight distinct digits are present).
  (iii) pathological cases: for example, a grid filled entirely with 7s.

- **Rule violation**: All digits 1–9 occur exactly nine times each, but the Sudoku constraints are violated. For instance, starting from a valid grid and swapping two adjacent digits in a row will typically introduce a column violation (and possibly a box violation as well). Here too, one can speak of a scale of deviation, depending on how many constraints are broken.

The class `GridCollection` offers the rather unusual convenience of letting you decide not only how many grids the garbage collection should contain, but also the degree and *kind* of deviance. An overview can be obtained from the `str` representation (e.g. via `print(gc); gc.getInfo()`):

```
GridCollection[  . . . . .
                 garbage: False
                   total size: 0
                     guest_grids              : 0
                     false_fromCurrent_seq    : 0
                     false_fromCurrent_switch : 0
                     false_cardinality        : 0
                     false_off_by_X           : 0
```

```
                    false_arbitrary          : 0
                . . . .
            ]
```

Alternatively, the same information can be retrieved via `gc.getGarbageStatus`.

At present, there is no compelling reason to include pathologically false grids, and consequently no functionality is provided for generating them. The following methods are available.

```
makeFalseGrids_arbitrary(how_many: int = 100000,
                         toGarbage: bool = True,
                         seed: Optional[int] = None,
                         check_explicitly: bool = False
                         ) -> Optional[np.ndarray]
```

Random generation. Based on `numpy.random.randint`, this method generates the most deviant class of false grids and may include all kinds of violations: rule violations, cardinality violations, and even missing digits. Parameters:

- `how_many`: number of grids to be generated.
- `toGarbage`: if set to `True` (default), the grids are added to the garbage collection.
- `seed`: random seed.
- `check_explicitly`: if `True`, the collection is explicitly checked for accidentally valid grids; otherwise (default), it is added directly to the garbage collection.
  Recall that there are $\approx 1.9 \times 10^{77}$ false grids versus only 6.7×10^{21} valid ones. By default, the method relies on randomness and the sheer vastness of the search space: the probability of accidentally generating a valid grid by random digit assignment is vanishingly small. Still, if this thought interferes with your sleep, feel free to set this parameter to `True`.

```
makeFalseGrids_cardinality(how_many: int = 100000,
                           toGarbage: bool = True,
                           seed: Optional[int] = None,
                           check_explicitly: bool = False
                           ) -> Optional[np.ndarray]
```

Random shuffling. This method respects cardinality—each digit occurs exactly nine times—but violates the Sudoku constraints in one or (more likely) several ways. The parameters behave as in the previous method. Regarding `check_explicitly`: the method starts from a valid grid and then shuffles it via random permutations of the grid contents. The probability of accidentally producing a valid grid is therefore higher than above, though still not something one would rationally bet on. Nevertheless, the option exists.

```
def makeFalseGrids_offBy_X(how_many: int = 100000,
                           toGarbage: bool = True,
                           X: int = 1,
                           how_far: int = 1,
                           ) -> Optional[np.ndarray]:
```

Increase value at random position(s). Starting from a grid that satisfies cardinality, this method manipulates the digit counts by selecting X random positions and increasing the corresponding values by how_far ((mod 8)+1). For example, with the default parameters $X = 1$ and $how\_far = 1$: if the selected position originally contains the value 3, it is replaced by 4. As a result, the count of digit 3 decreases to eight, while the count of digit 4 increases to ten. In this sense, the method allows for relatively fine-grained, local control over deviations from cardinality. Parameters:

- `X`: with $1 \leq X < 81$; number of values to be increased.
- `how_far`: offset added to the value(s) at the selected position(s).

The three methods discussed above operate purely on the basis of random generation. After the desired set of false grids has been created, these grids are added to the current garbage collection, thereby increasing its size.

In contrast, the following two methods operate directly on the active series—which therefore must be activated first. Both methods clear the current garbage collection before populating it anew; consequently, after their application one has

`gc.SIZE_garbageCollection == gc.SIZE_activeSeries`.

Additional false grids may, of course, be added afterwards.

```
makeFalseGrids_fromCurrent_seq(shift: int = 1):
```

Sequential overflow. This method iterates over the current active series and manipulates each grid as follows. Consider a sequential representation of the grid (e.g. `grid.flatten()`). All values are shifted one position to the right, i.e. their indices are increased by one; the final value at index 80 (which would otherwise fall outside the valid range) is reinserted at the now-vacant initial position, index 0.

The number of positional shifts can be increased. Note, however, that there is a catch: if `number_of_shifts % 27 == 0`, a valid grid is produced, since this operation is equivalent to a box-row permutation, which preserves validity. All other shift values lead to invalid grids.
Parameters:

- `shift`: with `shift % 27 != 0`, number of shifts "to the right".

`makeFalseGrids_fromCurrent_switch():`

Swap values. This method iterates over the current active series and manipulates each grid as follows. A random (valid) index $0 \leq idx < 80$ is selected, and the adjacent values are swapped via `grid[idx], grid[idx+1] = grid[idx+1], grid[idx]`. The method ensures that the two adjacent values are not identical before swapping—a nontrivial concern at row breaks, for example at $(r = 1, c = 9) : 7$ and $(r = 2, c = 1) : 7$. Swapping values in this manner automatically introduces a violation of the column (and possibly box) constraint. In short, this method injects a deliberately local rule violation.

```
make_guestGrids(how_many: int = 100000,
                series: int = 1)
```

Valid guests. Once this machinery is in place, it becomes clear that classification tasks need not be limited to validity alone. One may wish to test whether a grid has property A or not, or whether it belongs to grid family B, C, or D (or none of the above). For such scenarios, it can be useful for the garbage collection to contain *valid* grids as well.

This is precisely the purpose of this method: it creates valid grids (based on vertical permutation) and adds them to the garbage collection, where they receive the label 0 and are thus treated—formally, if not emotionally—like garbage. Currently, guest grids are generated from vertical series; other constructions are under development.

- `how_many`: number of grids to be generated.
- `series`: specifies the number of vertical series to be generated; from each series, $\lfloor \frac{how\_many}{series} \rfloor$ randomly selected grids are contributed to the prospective collection (see Sect. 1.1.4 for further comments on this parameter).

For binary classification, it is strongly recommended that the garbage collection be at least as large as the active series (see Sect. 4). For multi-class classification, the garbage collection must be at least as large as one class (currently: 362,880). Recall also that a garbage collection of the appropriate size must be initialized *before* activating the multi-class series; cf. Sect. 2.4.
Some illustrations; first, a plausible routine where the garbage collection is successively populated by a range of classes of flase grids:

```
>>> gc.activate_horizontalSeries()

>>> gc.makeFalseGrids_fromCurrent_switch()

>>> gc.makeFalseGrids_arbitrary(how_many=10000)
>>> gc.makeFalseGrids_cardinality(how_many=12345)
>>> gc.makeFalseGrids_offBy_X(how_many=9876, X=3, how_far=3)
>>> gc.make_guestGrids(how_many=8877, series=2)
```

```
>>> print(gc)
GridCollection[
                 internal shape: (373248, 81)
                 active series: horizontal
                    size: 362880
                 . . . . .
                 garbage: True
                   total size: 403978
                    guest_grids              : 8877
                    false_fromCurrent_seq    : 0
                    false_fromCurrent_switch : 362880
                    false_cardinality        : 12345
                    false_off_by_X           : 9876
                    false_arbitrary          : 10000
               . . . . .
              ]

>>> gc.clearGarbage()
```

Next an illustration of a contraproductive procedure; recall that the two methods `makeFalseGrids_fromCurrent_switch; makeFalseGrids_fromCurrent_seq` clear the garbage collection first. As a result, the garbage produced previously will be removed from the collection:

```
>>> gc.makeFalseGrids_arbitrary(how_many=10000)
>>> gc.makeFalseGrids_cardinality(how_many=12345)
>>> gc.makeFalseGrids_offBy_X(how_many=9876, X=3, how_far=3)
>>> gc.make_guestGrids(how_many=8877, series=2)

>>> gc.makeFalseGrids_fromCurrent_seq()

>>> print(gc)
GridCollection[ . . . . .
                 garbage: True
                   total size: 362880
                    guest_grids              : 0
                    false_fromCurrent_seq    : 362880
                    false_fromCurrent_switch : 0
                    false_cardinality        : 0
                    false_off_by_X           : 0
                    false_arbitrary          : 0
               . . . . .
              ]
```

4 Create Train/Test Datasets

The following three methods generate datasets based on the current active series and, where required, the garbage collection. All methods return the data as a tuple of arrays, following the familiar convention of `train_test_split` (from `sklearn`):

```
data = (X_train, X_test, y_train, y_test)
```

Each method involves randomization (shuffling) and stratification of the resulting datasets.

4.1 Binary Classification

```
split_binary(false_ratio: float = 0.5,
             train_ratio: float = 0.7,
             seed:        int   = None,
              ) -> Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]
-----------------------------------------------------------------------

>>> X_train, X_test, y_train, y_test = gc.split_binary()
```

This method combines the active series and (a selection from) the garbage collection into a single dataset. Grids from the active series are assigned the label 1, while grids drawn from the garbage collection receive the label 0.
Parameters:

- `false_ratio`: the proportion of false grids in the output datasets. When set to 0.5 (default), the dataset consists of 50% valid grids and 50% false grids (i.e. equally many valid and invalid examples). If the garbage collection is larger than the active series—which is always recommended—an appropriate number of false grids is randomly sampled from it.

  Since the space of false grids vastly exceeds that of valid ones, it can be reasonable to reflect this imbalance, at least partially, by increasing the false ratio; for example, `false_ratio = 0.7` corresponds to 30% valid and 70% false grids. Two points should be kept in mind:

  - The size of the active series is fixed, regardless of its proportion in the final dataset:
    `gc.SIZE_activeSeries` $\Leftrightarrow (1.0 - false\_ratio)\times$ size of the output dataset.
    Consequently, increasing the false ratio (i.e. decreasing the true ratio) necessarily increases the total size of the resulting dataset.
  - Accordingly, the garbage collection must be sufficiently large. More precisely, for a false ratio > 0.5, the number of false grids should be at least $\frac{false\_ratio}{true\_ratio} \times$ `gc.SIZE_activeSeries`.

- `train_ratio`: fraction of the data to be assigned to the training set.
- `seed`: random seed.